

MAVE Data Analysis I: Quantification of an SGE Experiment

What is Saturation Genome Editing (SGE)?

Saturation genome editing (SGE) is a massively parallel assay of variant effect (MAVE) that lets you functionally test thousands of variants in a single pooled experiment. Traditional approaches to understanding variant effects require testing mutations one at a time, which is slow and resource-intensive. SGE overcomes this limitation by using **CRISPR/Cas9** and **homology-directed repair (HDR)** to introduce entire libraries of predefined variants directly into an endogenous genomic locus in living cells.

The power of SGE lies in its ability to generate variants in their native genomic context, preserving the natural regulatory environment and chromatin structure. Once the variant library has been introduced into cells, you apply a selective pressure and measure the resulting changes in variant frequencies using deep sequencing. The type of selection determines which variants survive: in negative selection experiments (like the BAP1 study you'll work with), variants that disrupt essential gene function are depleted from the population over time, while benign variants remain stable or increase in frequency. In positive selection experiments (such as drug resistance screens), the pattern reverses, with disruptive variants potentially being enriched. By tracking these changes across multiple timepoints, you can derive quantitative functional scores for each variant.

This approach is particularly valuable for interpreting **variants of uncertain significance (VUS)** found in clinical sequencing, as it provides experimental evidence of functional impact rather than relying solely on computational predictions.

Learning Outcomes

By the end of this module, you will be able to:

- Explain how an SGE library is structured (targetons, primers, constant regions).
- Convert raw sequencing data (FASTQ files) into variant counts.
- Assess the quality of your data.
- Troubleshoot issues.

Pre-requisites

- Basic Unix command line skills (navigating directories, running commands).
- Familiarity with next-generation sequencing concepts (FASTQ files, paired-end reads).
- Some experience with R or Python for plotting (template code will be provided).

Dataset

For this practical, you'll use a subset of the sequencing data from Waters et al., focusing on the SGE experiments that target exon 5 of BAP1.

Waters, A.J., Brendler-Spaeth, T., Smith, D., Offord, V., Tan, H.K., *et al.*

Saturation genome editing of BAP1 functionally classifies somatic and germline variants.

Nature Genetics 56, 1434–1445 (2024).

[doi:10.1038/s41588-024-01799-3](https://doi.org/10.1038/s41588-024-01799-3) (<https://www.nature.com/articles/s41588-024-01799-3>)

Why BAP1?

BAP1 (BRCA1 Associated Protein 1) is a tumour suppressor gene that encodes a deubiquitinase enzyme involved in DNA repair, chromatin regulation, and cell cycle control. Germline loss-of-function variants cause an autosomal dominant cancer predisposition syndrome, increasing the risk of uveal melanoma, mesothelioma, renal cell carcinoma, and other cancers. Somatic BAP1 mutations are also frequently found in tumours.

The challenge: the majority of BAP1 variants observed clinically are missense changes classified as variants of uncertain significance (VUS), making it more difficult to provide accurate risk assessments and screening recommendations.

BAP1 Exon 5

Exon 5 sits within the N-terminal ubiquitin carboxy-terminal hydrolase (UCH) domain, the catalytic core of BAP1's deubiquitinase activity. This region harbours several clinically important germline variants linked to BAP1 tumour predisposition syndrome and abnormal splicing.

In the Waters paper, exon 5 served as a pilot region to optimise the SGE workflow in HAP1 cells before scaling up to all exons. The exon 5 library contains approximately 1,040 targetons representing all possible single-nucleotide variants across the target region, plus selected indels and other variant types. The dataset has been deposited in MaveDB as a standalone resource.

This makes exon 5 an ideal training locus. It's compact (a single approximately 245 bp target region), shows clear separation between disruptive and tolerated variants, and connects directly to real clinical questions about BAP1 variant interpretation.

What you'll do in this tutorial

For the tutorial, you'll work with downsampled FASTQ files and the corresponding VaLiAnT library design for exon 5. The full exon 5 dataset contains millions of reads, which would take too long to process during the practical session. We've downsampled the data to a manageable size that still retains sufficient coverage per targeton for you to see meaningful patterns and complete the tutorial within the time available.

The skills and workflow you learn here apply equally to other SGE experiments and exons, whether from the full BAP1 dataset or other genes entirely. In real-world applications, you would work with the full-depth sequencing data to maximise statistical power for variant scoring.

Go to the tutorial directory

Before we begin, you need to go the tutorial directory:

```
cd /home/manager/MAVE_2025/course_data_2025/mave_data_analysis_1
```

Exploring the dataset

Although quantification methods may differ in language or format, they typically require two inputs: sequencing data (reads) from your experiment and the library which defines your variants.

Let's examine the sequencing data first.

Sequencing data

The raw sequencing data for the Waters et al. BAP1 study is publicly available at the European Nucleotide Archive (ENA):

<http://ebi.ac.uk/ena/browser/view/PRJEB64778> (<http://ebi.ac.uk/ena/browser/view/PRJEB64778>)

QUANTS accepts data in either FASTQ or CRAM format. CRAM files are compressed (typically 50-70% smaller), but take longer to decompress during processing. For this tutorial we use FASTQ to save time, though sequencing facilities often deliver data in CRAM format.

For this tutorial, we've downloaded the sequencing data for exon 5 and downsampled it (reduced the number of reads per sample from >1M reads to 500,000 reads) so that QUANTS will run quickly for you. The read data is stored in FASTQ format in the data directory.

Let's take a look at our data:

```
ls data/exon5/fastq
```

Here you should see 30 FASTQ files (ending in `.fastq.gz`). Don't worry, we'll explain the experimental design and what each file represents soon!

Understanding FASTQ structure

FASTQ files contain sequencing reads in a specific format. Each read typically takes up 4 lines:

1. Header line (starts with @)
2. Sequence line (the actual DNA bases)
3. Plus line (starts with +, sometimes repeats the header)
4. Quality line (quality scores for each base)

For more information on FASTQ format, see https://en.wikipedia.org/wiki/FASTQ_format (https://en.wikipedia.org/wiki/FASTQ_format).

To see what that looks like, let's look at the first two reads in one of our files:

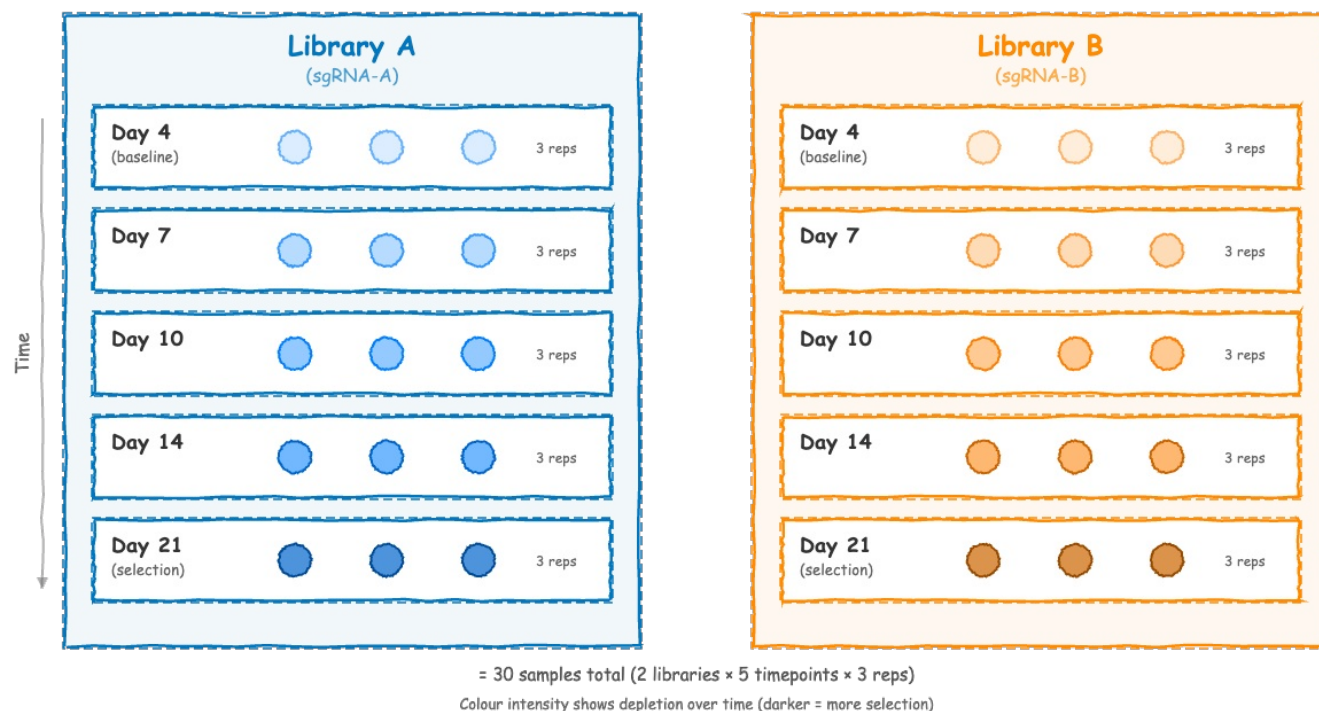
```
zcat data/exon5/fastq/5_a_Day4_Rep1_downsampled.fastq.gz | head -8
```

Experimental design

Each exon was edited with two independent SGE libraries (A and B), each using a different sgRNA. Cells were sampled at five timepoints after transfection (Days 4, 7, 10, 14 and 21), with three biological replicates at each timepoint.

Day 4 is your baseline, shortly after editing but before strong selection has occurred. By day 21, disruptive variants should have depleted as BAP1 is essential for cell survival. This allows you to compare variant frequencies across timepoints to reveal which are functional and which are disruptive. More on that later today!!

SGE Experimental Design: BAP1 Exon 5



Library design

The HDR template library was designed using VaLiAnT (Variant Library Annotation Tool), a tool for designing SGE libraries. The library file is a CSV (i.e. each column is comma-separated) containing metadata for every variant in the library.

VaLiAnT: <https://github.com/cancerit/VaLiAnT> (<https://github.com/cancerit/VaLiAnT>)

Barbon, L., Offord, V., Radford, E.J., Butler, A.P., Gerety, S.S., Adams, D.J., Tan, H.K., Waters, A.J. **Variant Library Annotation Tool (VaLiAnT): an oligonucleotide library design and annotation tool for saturation genome editing and other deep mutational scanning experiments.** *Bioinformatics* 38(4), 892-899 (2022).
[doi:10.1093/bioinformatics/btab776](https://doi.org/10.1093/bioinformatics/btab776) (<https://doi.org/10.1093/bioinformatics/btab776>)

Your VaLiAnT libraries for exon5 can be found in `data/valiant_libraries`. Let's take a look.

```
head data/exon5/valiant_library_5a.csv | column -t -s','
```

As you can see there is a lot of metadata. For our quantification, we just need a unique id for each of the represented mutations (oligos) and their sequence. In the VaLiAnT library these are the columns called `oligo_name` (column 1) and `mseq` (column 24) respectively.

Quantification

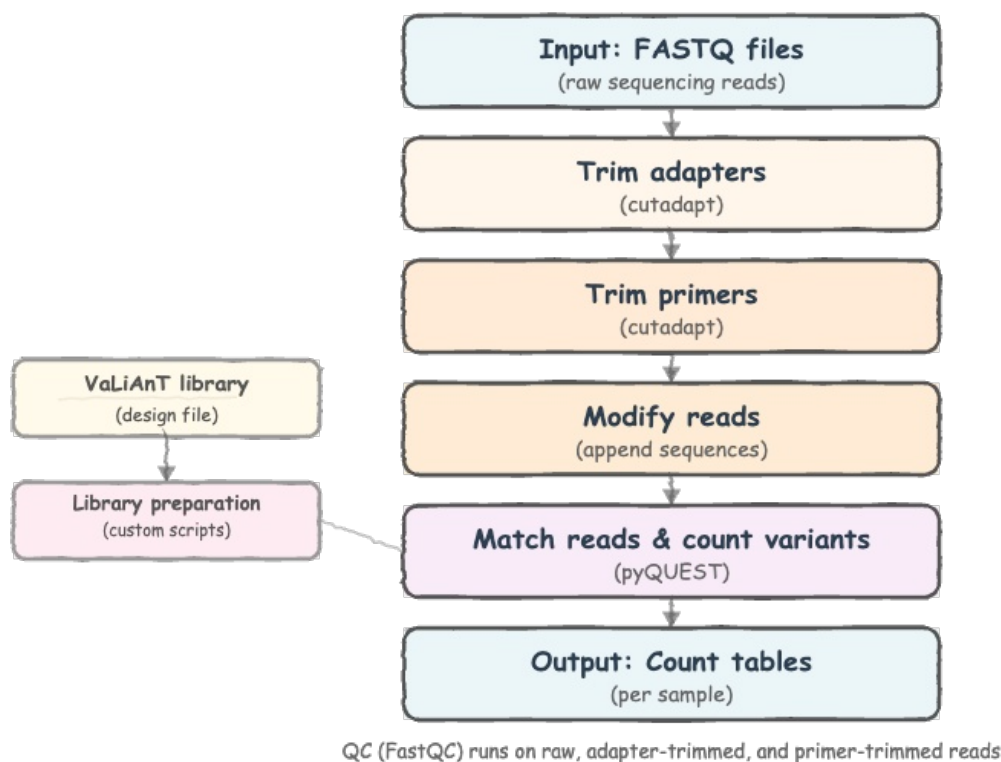
Quantification takes each sequence from your sample and checks if it matches one of the designed sequences in the library. It then counts how many times each designed sequence shows up, producing a count table of variant abundance per sample.

For this tutorial, you'll use QUANTS <https://github.com/cancerit/QUANTS> (<https://github.com/cancerit/QUANTS>), a Nextflow workflow specifically designed to convert FASTQ files from SGE experiments into count tables.

QUANTS is built using Nextflow, a workflow management system that chains together multiple processing steps. The pipeline incorporates existing bioinformatics tools (like cutadapt for trimming adapters and FastQC for quality control) along with custom scripts for matching reads to library sequences and generating counts. Each step runs in its own container (e.g. Docker), making the pipeline portable and reproducible across different computing environments.

The key advantage of QUANTS is that it handles all the steps from raw FASTQ files to counts automatically: trimming sequencing adapters, quality filtering, matching reads to your VaLiAnT library design, and outputting count tables. You provide the FASTQ files and library metadata, and QUANTS produces one count file per sample showing how many reads matched each designed variant.

An overview of how QUANTS works



In order for QUANTS to know where our data is and what to do with it we provide it with:

- a config file (which tools to run and their parameters)
- a samplesheet (mapping between our sample names and their corresponding FASTQ files)

QUANTS samplesheet

QUANTS needs to know which FASTQ files to process and how they're organised. You provide this information in a samplesheet which is a simple CSV (comma-separated) file that lists all your samples and their corresponding sequencing files.

```
cat data/exon5/samplesheet_5a.csv
```

Why is fastq_2 empty?

This dataset uses single-end sequencing, where only one end of each DNA fragment is sequenced. The fastq_2 column is left empty (just a comma with nothing after it). If you had paired-end data, you'd list the R2 files in this column - we'll look at an example in the bonus exercise.

Some simple rules

- Make sure you have all three columns (even if the values for fastq_2 are left blank)
- Make sure you have these header names (sample_name, fastq_1, fastq_2) because they are used by QUANTS (for validation)
- Make sure file paths are correct relative to where you'll run QUANTS
- Check that sample names are unique
- Ensure there are no spaces in the sample names

QUANTS will process each sample listed in the samplesheet, running the trimming and quantification steps on all files automatically.

QUANTS configuration

QUANTS is a highly configurable workflow. We're not going to go through all of the options here, but there's more information in the QUANTS GitHub repository if you're interested!

When you run QUANTS, you provide a configuration file that tells the pipeline how to process your data. This file is written in JSON (JavaScript Object Notation), a simple text format for storing structured data. JSON uses key-value pairs like "parameter_name": "value", where values can be numbers, text (in quotes), true/false, or null (meaning no value).

Let's start by looking at the configuration file (config) for this experiment.

```
cat data/exon5/quant_5a.json
```

Understanding the parameters

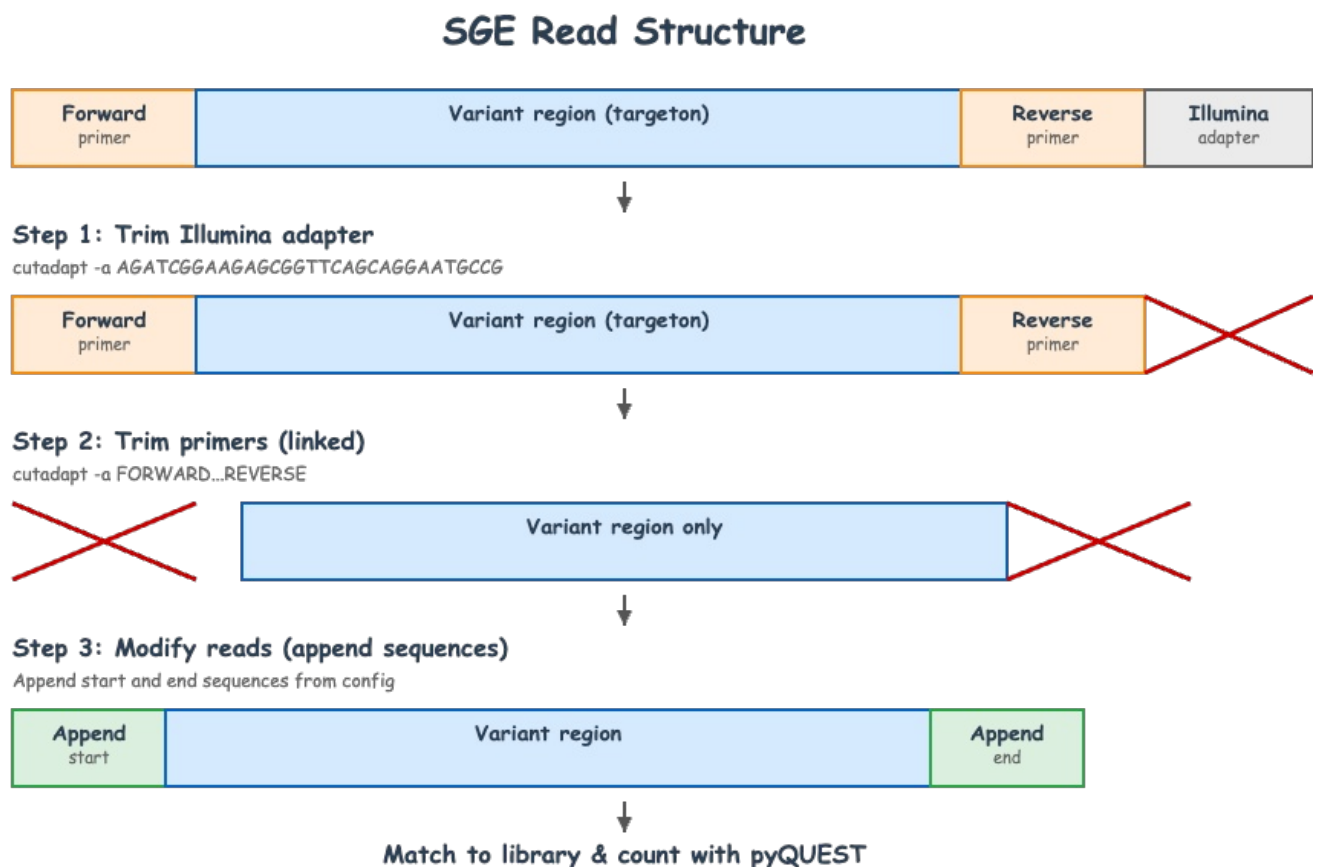
Below is a brief explanation of each of the parameters:

Parameter	Value	Description
max_cpus	4	Number of processors to use for parallel processing
single_end	true	Reads are single-end (not paired-end sequencing)
input_type	fastq	Input files are in FASTQ format
raw_sequencing_qc	true	Generate quality control reports for raw sequencing files
adapter_trimming	cutadapt	Use cutadapt to remove Illumina adapters
adapter_trimming_qc	true	Generate quality control reports for adapter trimming step
adapter_cutadapt_options	-a AGATCGGAAGAGCGGTTCAGCAGGAATGCCG	Cutadapt command to remove 3' Illumina adapter sequence from reads
primer_trimming	cutadapt	Use cutadapt tool to remove primer sequences
primer_trimming_qc	true	Generate quality control reports for primer trimming step
primer_cutadapt_options	-a GGATCCCCATTCTTGATGTATATGGGC...CTACCACATGATATTGGGTACTATT	Cutadapt command to remove linked 3' primers (forward...reverse) from reads

read_modification	true	Allow modification of read sequences
append_start	AATGATACGGCGACCACCGA	Constant sequence to add at the 5' end (after the forward primer position)
append_end	TCGTATGCCGTCTTCTGCTTG	Constant sequence to add at the 3' end (before the reverse primer position)
append_quality	?	Quality score for appended bases
transform_library	true	Convert VaLiAnT library format to pyQUEST-compatible format
pyquest_library_converter_options	-N 1 -S 24	Converter options: id in column 1 (oligo_name), sequence in column 24 (mseq)
quantification	pyquest	Use pyQUEST for matching reads to library and counting

Why do we need to remove adapters and primers from our reads?

After sequencing, your reads may contain more than just the variant sequence you want to quantify. This can differ between experiments. The structure of the reads from our BAP1 exon 5 (guide A) experiment looks like this:



For quantification, only the variant region (the actual edited sequence) should be matched against your exon5a VaLiAnT library sequence. Everything else needs to be removed.

What are these extra sequences?

The **Illumina adapter** (AGATCGGAAGAGCGGTTCAGCAGGAATGCCG) is a standard sequencing adapter added during library preparation. It allows DNA fragments to bind to the flow cell for sequencing and has nothing to do with your variants.

The **primers** are constant sequences that flank the variant region:

Forward primer: GGATCCCCATTCTTGATGTATATGGGC Reverse primer: CTACCACATGATATTGGGTACTATT

They're the same across all variants in your library, so they don't help identify which specific variant is present.

Why trim in this order?

QUANTS removes sequences in two steps:

Step 1: Trim Illumina adapter

```
adapter_cutadapt_options: "-a AGATCGGAAGAGCGGTTCAGCAGGAATGCCG"
```

The Illumina adapter is removed first because it's at the very end of the read and may not always be present in full (if the read is shorter than the insert). Removing it first prevents it from interfering with primer detection.

Step 2: Trim homology arm primers

```
primer_cutadapt_options: "-a GGATCCCCATTCTTGATGTATATGGGC...CTACCACATGATATTGGGTACTATT"
```

The ... notation tells cutadapt these are linked primers so that it looks for the forward primer at the start and the reverse primer at the end, then removes both. This leaves only the variant region in the middle.

After both trimming steps, you're left with clean variant sequences that can be matched against the `mseq` sequences in your VaLiAnT library to count how many times each variant appears.

Technical note: The -a flag in cutadapt specifies a 3' adapter (sequence at the end of the read). For linked primers, cutadapt searches for both the forward sequence at the 5' end and reverse sequence at the 3' end. For more information see the cutadapt manual <https://cutadapt.readthedocs.io/en/stable/> (<https://cutadapt.readthedocs.io/en/stable/>).

Why do we need to add constant sequences to our reads?

The VaLiAnT library file contains the complete oligonucleotide sequences (in the `mseq` field), but after adapter and primer trimming, your sequencing reads contain only the variant region. To match reads to the library, we need to reconstruct the full sequence by adding back the constant flanking sequences.

- `append_start` and `append_end` : Constant sequences included in the library design which sit outside of the primers
- `append_quality` : Since we're adding synthetic sequences (not actually sequenced), we need to assign them quality scores. The ? character represents a Phred quality score of 30 (Q30), indicating 99.9% base call accuracy - a reasonable default for known sequences.

Not all SGE experiments require read modification - it depends on your library design.

Running QUANTS

Now that you have your samplesheet, configuration file, and input data ready, you can run QUANTS using Nextflow!

Why use absolute paths?

Before running the command, it's important to understand why we use `${PWD}` (Present Working Directory). When Nextflow runs processes inside Docker containers, the containers have their own isolated filesystem. For the container to access files on your computer, Nextflow mounts your directories into the container.

Using relative paths (like `data/exon5/samplesheet.csv`) can cause problems because:

- The container might not know where to find the files
- Docker volume mounts need absolute paths to work reliably
- Different steps might run in different working directories

Using absolute paths with `${PWD}` ensures:

- Files are always accessible regardless of where the container is running
- All paths are explicit and unambiguous
- Docker can properly mount the necessary directories

`${PWD}` expands to your current working directory's full path (e.g., `/home/manager/my_full_path/mave_data_analysis_1`).

Why split the command across multiple lines?

The QUANTS command is long and can be hard to read on a single line. We use the backslash `\` at the end of each line to split the command across multiple lines.

```
nextflow run /home/manager/QUANTS/main.nf \
-c ${PWD}/data/exon5/nextflow.config \
-params-file ${PWD}/data/exon5/quant_5a.json \
--input ${PWD}/data/exon5/samplesheet_5a.csv \
--oligo_library ${PWD}/data/exon5/valiant_library_5a.csv \
--outdir ${PWD}/results_5a
```

The backslash `\` is a line continuation character in bash. It tells the shell "this command continues on the next line." This makes the command:

- Easier to read and understand
- Simpler to edit individual parameters
- Less prone to typos

Important: There must be no spaces after the backslash, and the backslash must be the last character on the line. Otherwise, the command won't work.

You could also write this as a single line (all one command with no backslashes), but we haven't done that here as it doesn't render in the tutorial document very well.

The QUANTS command explained

The first part of the command, `nextflow run /home/manager/QUANTS/main.nf`, tells Nextflow to execute the QUANTS pipeline. Let's break this down:

- `nextflow run` : This is the Nextflow command to execute a workflow
- `/home/manager/QUANTS/main.nf` : This is the path to the main workflow file for QUANTS

The `main.nf` file is the entry point for the QUANTS pipeline. It's written in Nextflow's DSL (Domain Specific Language) and contains:

- The overall structure of the pipeline (which steps to run and in what order)
- References to all the individual process modules (FastQC, cutadapt, pyQUEST, etc.)
- Logic for how data flows between processes

You don't need to know how Nextflow works in details for this tutorial, we'll give you all the commands you need and explain the outputs. It's more important we look at the data going in, broadly what it's doing to the data and what data comes back out!

Here's a quick explanation of the information we're giving QUANTS:

Option	Value	Description
-c	\${PWD}/data/exon5/nextflow.config	Nextflow configuration (Docker settings, resource limits)
-params-file	\${PWD}/data/exon5/quant_5a.json	Pipeline parameters (trimming options, tools to use)
--input	\${PWD}/data/exon5/samplesheet_5a.csv	Samplesheet listing all FASTQ files to process
--oligo_library	\${PWD}/data/exon5/valiant_library_5a.csv	ValiAnT library design file
--outdir	\${PWD}/results_5a	The results directory for final output files

```
nextflow run /home/manager/QUANTS/main.nf \
-c ${PWD}/data/exon5/nextflow.config \
-params-file ${PWD}/data/exon5/quant_5a.json \
--input ${PWD}/data/exon5/samplesheet_5a.csv \
--oligo_library ${PWD}/data/exon5/valiant_library_5a.csv \
--outdir ${PWD}/results_5a
```

QUANTS will take approximately 20 minutes to process all 15 samples.

Understanding QUANTS Outputs

QUANTS creates several directories and files during execution.

The `.nextflow.log` file

This is a detailed log file created in your launch directory. It contains:

- Detailed execution information for every process
- Error messages and stack traces if something fails
- Resource usage (CPU, memory)
- File paths and commands executed

This file is essential for debugging when things go wrong.

The `work` directory

Nextflow's temporary working directory where all intermediate files are stored. Each task gets a unique subdirectory (identified by hash codes like `8b/2191ed...`).

This directory:

- Contains all intermediate files from each processing step
- Can be safely deleted after the pipeline completes successfully
- Is useful for debugging failed tasks
- Can be very large (several GB)

The `results_5a` directory

Your final results organised by analysis type:

```
results_5a/
├── cutadapt/           # Adapter and primer trimming logs
├── fastqc/             # Quality control reports (HTML files)
├── modified_fastq/     # Trimmed reads with constant sequences appended
├── multiqc/            # Combined QC report (multiqc_report.html)
├── pyquest/            # Count tables (one per sample)
├── seqkit_stats/       # Read statistics
└── pipeline_info/      # Execution reports and metadata
```

For more detailed information please see:

Nextflow documentation: <https://www.nextflow.io/docs/latest/> (<https://www.nextflow.io/docs/latest/>)

Nextflow debugging guide: <https://www.nextflow.io/docs/latest/tracing.html> (<https://www.nextflow.io/docs/latest/tracing.html>)

QUANTS documentation: <https://github.com/cancerit/QUANTS> (<https://github.com/cancerit/QUANTS>)

Exercise 1: Checking Adapter Trimming Efficiency

After adapter trimming, it's important to know how many reads had adapters and were successfully trimmed.

Navigate to `results_5a/cutadapt` and the log file for Day 4 Replicate 1 (i.e. `5_a_Day4_Rep1.adapter.cutadapt.log`).

```
head results_5a/cutadapt/5_a_Day4_Rep1.adapter.cutadapt.log
```

Look for these key lines:

- Total reads processed
- Reads with adapters
- Reads written (passing filters)

Questions

1.1) How many reads were processed by cutadapt (i.e. how many reads were in the raw sequencing file we gave it)?

1.2) What percentage of reads contained the Illumina adapter sequence?

Compare this across several samples, are the adapter trimming rates consistent?

Troubleshooting tip: if you get a low proportion of reads with adapter sequences, it may be correct and you have none but it can also be that either your adapter sequence or the adapter trimming parameters are incorrect. For low quality sequences, you can also relax the matching parameters.

Exercise 2: Checking Primer Trimming Success

After primer trimming, we want to confirm that both forward and reverse primers were found and removed.

Navigate to `results_5a/cutadapt` and the log file for Day 4 Replicate 1 (i.e. `5_a_Day4_Rep1.primer.cutadapt.log`).

```
head -16 results_5a/cutadapt/5_a_Day4_Rep1.primer.cutadapt.log
```

Look for these key lines:

- Total reads processed
- Reads with adapters
- Total basepairs processed

Questions

2.1) What percentage of reads contained both the forward and reverse primer?

2.2) What reason might there be for some reads to not be trimmed?

Compare this across several samples, are the primer trimming rates consistent?

Troubleshooting tip: if you get a low proportion of reads with primer sequences, it may be correct and you have none but it can also be that either your primer sequence(s) or the primer trimming parameters are incorrect. For low quality sequences, you can also relax the matching parameters.

Exercise 3: MultiQC Quality Overview

MultiQC combines statistics from all tools and samples into one comprehensive report. Open the MultiQC report at `results_5a/multiqc/multiqc_report.html` in a web browser.

Questions

3.1) How long were our raw reads?

3.2) At what position (bp) does adapter content start to appear?

3.3) What percentage of reads contain adapter sequences by the end of the read?

3.4) What percentage of reads contain adapter sequences after adapter trimming has taken place?

3.5) What is the average read length after adapter trimming and after primer trimming? Why are the reads not all the same length after trimming?

Exercise 4: Understanding the pyQUEST count files and statistics

pyQUEST matches trimmed and modified reads to the library provided and quantifies how many times each library oligo appears in each sample.

pyQUEST generates three output files per sample:

File	Description	Use
[sample].stats.json	Summary statistics about read mapping and variant coverage	Quality control and troubleshooting
[sample].lib_counts.tsv.gz	Library-dependent counts - abundance of each designed library variant	Main output for downstream analysis
[sample].query_counts.tsv.gz	Library-independent counts - abundance of all unique read sequences found	Debugging and identifying off-target sequences

For analysing your SGE experiment, you'll work with the `lib_counts.tsv.gz` files. These contain counts for your designed oligos (matching the identifiers from your library). The `query_counts.tsv.gz` files are useful for troubleshooting if you have low mapping rates or want to see what other sequences are present.

Part A: Interpreting pyQUEST Statistics

Let's start by examining the statistics file to understand how well the quantification worked. The `stats.json` files contain some useful statistics:

Statistic	Meaning
input_reads	Total reads after trimming and modification
mapped_to_template_reads	Reads that matched a library oligo
unmapped_reads	Reads that didn't match any library oligo
multimap_reads	Reads that matched multiple oligos (ambiguous)
mean_count_per_template	Average reads per oligo
median_count_per_template	Median reads per oligo (less affected by outliers)
total_templates	Total oligos in library (including redundant ones)
total_unique_templates	Unique functional oligos
zero_count_templates	Oligos with no reads detected
low_count_templates_lt_15	Oligos with <15 reads (low coverage)
gini_coefficient	Measure of coverage inequality (0=perfect equality, 1=extreme inequality)

First, let's look at the JSON statistics file that pyQUEST has generated for Day 4 Replicate 1.

```
jq '. ' results_5a/pyquest/5_a_Day4_Rep1.stats.json
```

Questions

4.1) What percentage of input reads successfully mapped to library variants?

Calculate: $(\text{mapped_to_template_reads} / \text{input_reads}) \times 100$

4.2) How well is the library represented at Day 4?

Look at:

- `zero_count_templates` : How many variants have no reads?
- `low_count_templates_lt_15` : How many variants have <15 reads?
- `low_count_templates_lt_30` : How many variants have <30 reads?

What does this tell you about library coverage at the baseline timepoint (Day 4)?

4.3) What does the mean vs. median tell you about the distribution?

- `mean_count_per_template` : 243.25
- `median_count_per_template` : 174.0

The mean is higher than the median. What does this suggest about how reads are distributed across variants?

Now view the statistics for Day 21 Replicate 1.

```
jq '. results_5a/pyquest/5_a_Day21_Rep1.stats.json'
```

Compare the two timepoints and answer:

4.4) How has the library coverage changed by Day 21?

Create a comparison table:

Metric	Day 4	Day 21	Change
<code>zero_count_templates</code>			
<code>low_count_templates_lt_15</code>			
<code>low_count_templates_lt_30</code>			
<code>gini_coefficient</code>			

What biological process explains these changes?

4.5) How has the mapping rate changed? Why?

Metric	Day 4	Day 21
<code>input_reads</code>		
<code>mapped_to_template_reads</code>		
Percentage reads mapped		

The mapping rate increased from Day 4 to Day 21. Why might this happen during negative selection?

Part B: Exploring Query Counts (All Unique Sequences)

The `query_counts.tsv.gz` file shows all unique sequences found in your sample, not just those matching your library. This is useful for understanding what's in your data.

Let's look at the structure of the query counts file:

```
zcat results_5a/pyquest/5_a_Day4_Rep1.query_counts.tsv.gz | head -5
```

The query_counts file is tab-separated with three columns:

1. **SEQUENCE** - The unique read sequence found in your sample
2. **LENGTH** - Length of the sequence in base pairs
3. **COUNT** - Number of times this exact sequence was observed

Each row represents a unique sequence detected, regardless of whether it matches your library.

Questions:

4.6) What are the two most abundant read sequences in Day 4 Rep1?

Use this command to find them:

```
zcat results_5a/pyquest/5_a_Day4_Rep1.query_counts.tsv.gz | tail -n +2 | sort -k3 -n -r | head -5 | column -t
```

```
zcat results_5a/pyquest/5_a_Day21_Rep1.query_counts.tsv.gz | tail -n +2 | sort -k3 -n -r | head -5 | column -t
```

This command chains together several steps using pipes (|):

1. `zcat` - decompress and read the file
2. `tail -n +2` - skip the header lines
3. `sort -k3 -n -r` - sort by column 3, numerically, highest first (i.e. sort by highest count)
4. `head -5` - take the top 5 lines
5. `column -t` - format as a neat table

Result: Shows the 5 most abundant sequences in your sample.

Looking for the most abundant reads can help debug when quantification goes wrong. But they can also represent two key sequences.

Special sequences

There are two special sequences you should be aware of:

PAM sequence: This is the sequence containing the PAM (Protospacer Adjacent Motif) without any edits. In SGE, you typically include PAM protection edits (PPEs) in your variants to prevent Cas9 from re-cutting after HDR. The unedited PAM sequence represents cells where:

- HDR failed and the original sequence remains
- The PAM wasn't protected, so Cas9 could re-cut
- Non-homologous end joining (NHEJ) created the exact PAM sequence

A **high PAM count** suggests low HDR efficiency or problems with PAM protection strategy.

Another sequence type to be aware of is your 'REF' sequence.

REF sequence: This is the wild-type (reference genome) sequence for this region. It represents:

- Unedited cells (HDR didn't occur)
- Cells where editing failed
- Background wild-type sequence

The **REF count** gives you a baseline for editing efficiency. In a well-edited library at Day 4, you expect:

- Relatively low REF counts (most cells should have variants)
- REF counts might increase over time if the wild-type is functional
- Very high REF counts at Day 4 might indicate poor editing efficiency

We extracted the `ref_seq` and `pam_seq` from the VaLiAnT library, reverse complemented them (since the library is on the minus strand), and added the append sequences to match the processed read structure. We then searched for these exact sequences in the query_counts files.

REF:

```
AATGATACGGCGACCAACCGATTGGGGCTTGCACTGAGGGGTGCTGTGTATGGGTGACTATTCTTGGTTTCACAGCTGATACCCAACCTCTTGCAACTC  
ATGCCTTGCTGAGCGTGCTCTGAACCTGCAGCAGCGTGACCTGGGACCCACCTGAGTCGCATGAAGGACTTCACCAAGGGTTTCAGCCCTGAGGTAG  
GCTGCAGTGCCCTTCATCTGGCTCACAGCCAACCTGGGCAGATCTGACCCTGAGGGCCACTGGGAATGTCGTATGCCGTCTTCTGCTTG
```

PAM:

AATGATACGGCGACCACCGATTGGGGCTTGCAGTGAGGGGTGCTGTGTATGGGTGACTATTCTTGTTTCACAGCTGATACCCAACCTCTTGTGCAACTC
ATGCTTTGCTAAGCGTGCTCCTGAACTGCAGCAGCGTGGACCTGGGACCCACCCTGAGTCGCATGAAGGACTTCACCAAGGGTTTCAGCCCTGAGGTAG
GCTGCAGTGCCTTCATCCTGGCTCACAGCCAACCTGGGCAGATCTGACCCTGAGGGCCACTGGGAATGTCGTATGCCGTCTTCTGCTTG

Using the `query_counts.tsv.gz` and the `stats.json` per sample, it's possible to calculate the percentage of all reads in a sample which are comprised of REF and PAM sequences.

Sample	REF Count	PAM Count	Total Reads	% REF	% PAM
5_a_Day4_Rep1	14,017	56,705	498,573	2.8%	11.4%
5_a_Day4_Rep2	13,895	56,321	498,499	2.8%	11.3%
5_a_Day4_Rep3	13,354	54,640	498,665	2.7%	11.0%
5_a_Day7_Rep1	8,046	63,183	498,798	1.6%	12.7%
5_a_Day7_Rep2	7,183	63,445	498,711	1.4%	12.7%
5_a_Day7_Rep3	11,087	62,334	498,655	2.2%	12.5%
5_a_Day10_Rep1	5,183	74,426	498,732	1.0%	14.9%
5_a_Day10_Rep2	3,821	75,254	498,821	0.8%	15.1%
5_a_Day10_Rep3	7,800	73,752	498,735	1.6%	14.8%
5_a_Day14_Rep1	4,286	97,191	498,815	0.9%	19.5%
5_a_Day14_Rep2	2,882	95,285	498,815	0.6%	19.1%
5_a_Day14_Rep3	8,061	94,641	498,832	1.6%	19.0%
5_a_Day21_Rep1	1,817	114,261	498,919	0.4%	22.9%
5_a_Day21_Rep2	1,145	111,594	498,856	0.2%	22.4%
5_a_Day21_Rep3	3,213	112,440	498,854	0.6%	22.5%

The PAM-protected sequence contains synonymous mutations that preserve BAP1 function, allowing cells to survive and become enriched as disruptive variants deplete from the population. The low starting percentage (~3%) indicates good HDR efficiency, and the steady depletion likely represents unedited cells that underwent Cas9 cutting followed by disruptive NHEJ-mediated indels.

Exercise 5: Exploring Library Counts

The `lib_counts.tsv.gz` files contain counts for each designed library variant - this is your main quantification output.

```
zcat results_5a/pyquest/5_a_Day4_Rep1.lib_counts.tsv.gz | head
```

The lib_counts file structure:

The first two comment lines (starting with ##) contain the pyQUEST command that was run and the version of pyQUEST used. Each subsequent row represents one designed library variant with its count in that sample.

Column	Name	Description
1	ID	Numeric identifier for the variant
2	NAME	Oligo name from VaLiAnT library (contains genomic coordinates and variant info)
3	SEQUENCE	Matched oligo sequence
4	LENGTH	Sequence length in base pairs
5	COUNT	Number of reads matching this oligo
6	UNIQUE	Whether this sequence is unique (1) or redundant (>1) in the library
7	SAMPLE	Sample name

Questions

5.1) What version of pyQUEST was used to generate these counts?

5.2) What is the name and length of the first oligo in the count table and how abundant was it? Is it a unique sequence?

5.3) What can you infer from the counts for the variant ENST00000460680.6.ENS00000163930.10_chr3:52408050_1del_rc ?

Let's take a look at the counts for this variant at each timepoint.

```
VARIANT='ENST00000460680.6.ENS00000163930.10_chr3:52408050_1del_rc'

for file in results_5a/pyquest/*.lib_counts.tsv.gz; do
    sample=$(basename $file .lib_counts.tsv.gz)
    count=$(zgrep "$VARIANT" $file | cut -f5)
    echo "$sample: $count"
done
```

Summary

Congratulations! You've successfully completed the SGE quantification tutorial. Let's review what you've learned.

What You've Accomplished

In this tutorial, you have:

- ✓ **Understood SGE fundamentals** - You learned how saturation genome editing works and why it's valuable for interpreting variants of uncertain significance
- ✓ **Explored real experimental data** - You examined the BAP1 exon 5 dataset structure, including FASTQ files and VaLiAnT library designs
- ✓ **Run a quantification pipeline** - You used QUANTS to convert raw sequencing reads into variant counts through:
 - Adapter trimming (removing Illumina adapters)
 - Primer trimming (removing homology arm primers)
 - Read modification (reconstructing full sequences for matching)
 - Variant quantification (matching reads to library and counting)
- ✓ **Assessed data quality** - You learned to:
 - Interpret cutadapt trimming logs
 - Evaluate adapter and primer detection rates
 - Use MultiQC reports to check quality across all samples
 - Understand pyQUEST mapping statistics
- ✓ **Observed selection in action** - By comparing Day 4 and Day 21 statistics, you saw evidence of negative selection as disruptive BAP1 variants became depleted from the population.

Checking Your Answers

If you'd like to compare your answers to the exercise questions, refer to the **answers PDF** ([.answers.pdf](#)).

Further Resources

Documentation and tools:

- QUANTS: <https://github.com/cancerit/QUANTS> (<https://github.com/cancerit/QUANTS>)
- VaLiAnT: <https://github.com/cancerit/VaLiAnT> (<https://github.com/cancerit/VaLiAnT>)
- Nextflow: <https://www.nextflow.io/> (<https://www.nextflow.io/>)
- cutadapt: <https://cutadapt.readthedocs.io/> (<https://cutadapt.readthedocs.io/>)

Key publications:

- Waters et al. (2024) - BAP1 SGE study: [doi:10.1038/s41588-024-01799-3](https://doi.org/10.1038/s41588-024-01799-3) (<https://doi.org/10.1038/s41588-024-01799-3>)
- Barbon et al. (2022) - VaLiAnT tool: [doi:10.1093/bioinformatics/btab776](https://doi.org/10.1093/bioinformatics/btab776) (<https://doi.org/10.1093/bioinformatics/btab776>)

Community resources:

- MaveDB - repository of MAVE datasets: <https://www.mavedb.org/> (<https://www.mavedb.org/>)
 - Atlas of Variant Effects Alliance: <https://www.varianteffect.org/> (<https://www.varianteffect.org/>)
-

Troubleshooting Tips

If you encounter issues with your own SGE data:

Low adapter/primer detection

- Verify adapter/primer sequences are correct
- Check if sequences need reverse complementing
- Try relaxing matching parameters in cutadapt

Low mapping rates

- Verify library file matches your sequencing data
- Check read modification parameters (append sequences)
- Examine query_counts to see what sequences are present
- Consider whether your library needs transformation

High variability between replicates

- Check for sample swaps (look at correlation between replicates)
- Review QC metrics - one replicate may have failed
- Consider sequencing depth - low coverage can cause noise

Many zero-count variants at baseline

- May indicate poor library representation
- Check if library was properly amplified
- Review sequencing depth - may need more reads

Well done on completing this tutorial!